

TECHNICAL DEEP DIVE

Jobflow Project Deep Dive

A visual A→Z explanation of architecture, code, data flow, and engineering decisions

Documentation package

Prepared from the Jobflow V3 codebase and current product direction.

Jobflow is a Django-based job application tracking workspace designed to turn scattered applications into a clear, actionable pipeline.

Contents

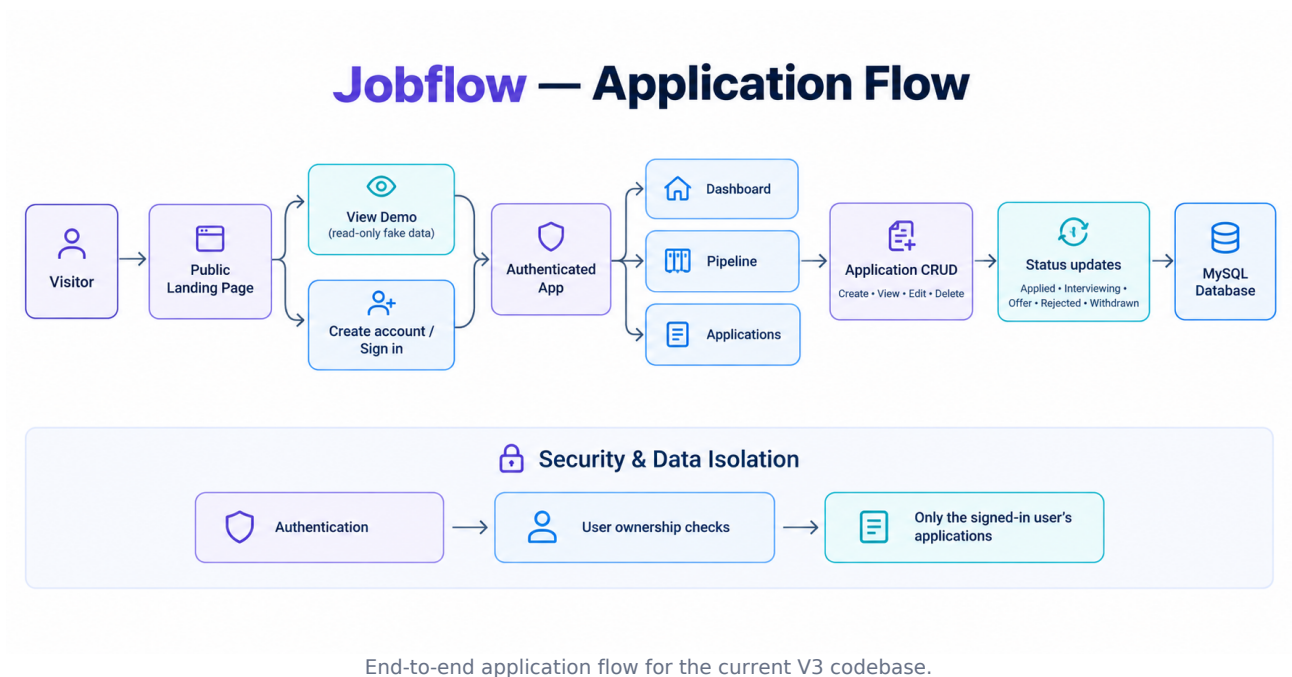
Section	What it covers
Product idea	Problem, scope, design philosophy
Architecture	Application flow and boundaries
Stack	Django, MySQL, JavaScript
Code structure	Project/app responsibilities
Data model	Fields and relationships
Request flow	Browser → view → ORM → template
Security	Authentication and ownership
Frontend	Interactions and motion
Production	Readiness roadmap
Takeaways	What the project demonstrates

1. Product idea

Job applications become difficult to manage when they are spread across spreadsheets, bookmarks, email, and memory. Jobflow turns each application into a structured record and places those records inside a workflow.

The product model is intentionally small: a user owns many applications; each application has a status, dates, optional job URL, and notes.

2. Architecture at a glance



3. Technology stack

Layer	Technology	Role
Web framework	Django	Routing, business logic, forms, authentication, ORM, templates
Frontend	Django templates + CSS + vanilla JS	UI rendering and browser interactions
Database	MySQL	Users and applications in current local setup
Driver	PyMySQL	Python MySQL connectivity
Static	WhiteNoise	Static asset serving
Server	Gunicorn	Production WSGI server

4. Code structure

job_tracker/ contains settings, root routing, and WSGI configuration. tracker/ contains the model, forms, views, URLs, templates, and static assets.

5. Data model

Application fields: owner, company, role, status, date_applied, next_action_date, job_url, notes, created_at, and updated_at. Owner is a foreign key to Django's User model.

Field	Purpose
owner	Links the record to one authenticated user
company / role	Identifies the opportunity
status	One of five controlled workflow states
date_applied	Original application date
next_action_date	Interview, deadline, or follow-up
job_url	Optional external listing
notes	Free-form context
timestamps	Created/updated tracking

6. Request flow

Normal request: browser → URL resolver → view → ORM query → template context → HTML response.
Status movement uses asynchronous JSON POST and returns JSON after validation and save.

Status update pattern

```
application = get_object_or_404(Application, pk=pk, owner=request.user)
new_status = payload.get("status")
if new_status not in valid_statuses: return HttpResponseBadRequest(...)
application.status = new_status
application.save(update_fields=["status", "updated_at"])
```

7. Frontend interactions

- HTML5 drag-and-drop moves cards.
- fetch() sends asynchronous status updates.
- Client-side search filters cards and table rows instantly.
- Dashboard counters animate with requestAnimationFrame().
- IntersectionObserver provides reveal motion and respects reduced-motion preferences.
- Ctrl/Cmd + K or / focuses search.

8. Authentication & authorization

Django handles authentication. Private views use @login_required. Application lookups also use owner=request.user, preventing a user from editing, deleting, or changing another user's record by guessing a primary key.

9. Public vs private

V3 separates the public experience (/ and /demo/) from the real tracker (/app/). This creates a clean portfolio entry point and a safe demonstration path.

10. Dashboard logic

The dashboard counts each status, serializes chart data, selects next actions from today through the next 14 days, and exposes the five most recent records.

11. Design system

The UI uses a premium dark SaaS direction with strong typography, purple/cyan accents, rounded cards, badges, motion, responsive navigation, empty states, and application metadata.

12. Production roadmap

- Move secrets to environment variables.
- Set `DEBUG=False` and configure `ALLOWED_HOSTS`.
- Collect and serve static files with `WhiteNoise`.
- Use a managed production database and `HTTPS`.
- Run behind `Gunicorn`.
- Add backups, logging, monitoring, error pages, and automated tests.

13. Engineering takeaway

Jobflow demonstrates a pragmatic full-stack approach: one Django application can provide routing, backend logic, authentication, database access, server-rendered UI, asynchronous interactions, and polished frontend behavior without requiring a separate SPA.